

Swing Components

Swing is a part of Java Foundation Classes (JFC) and provides a rich set of GUI components for developing platform-independent window-based applications.

Unlike the Abstract Window Toolkit (AWT), Swing components are lightweight, pluggable, and platform-independent because they are written entirely in Java. Swing supports highly customizable components like buttons, text fields, lists and panels.

Below are the most commonly used Swing Components:

1. JLabel - display a short string or image; cannot be edited by user

example:

```
JLabel label = new JLabel("Welcome!");
```

2. JButton - A push button that performs an action when clicked.

```
JButton button = new JButton("Click here");
```

3. JTextField - Allows the user to enter a single line of text

```
JTextField textField = new JTextField(20);
```

4. JTextArea - Allows multiple line of editable text.

```
JTextArea textArea = new JTextArea(5, 20);
```

5. JCheckBox - Lets the user select or deselect an option. Multiple checkboxes can be selected at once.

```
JCheckBox checkBox = new JCheckBox("Accept Terms");
```

6. JRadioButton - Allows user to choose one option from a group.

```
JRadioButton radio1 = new JRadioButton("Male");
```

7. JComboBox - A drop down list of options. The user can choose one from the list. Example:

```
JComboBox<String> country = new JComboBox<> (new String[]  
    { "Bangladesh", "Pakistan", "Afghanistan" });
```

8. JList - Displays a list of items for multiple selections.

```
JList<String> list = new JList<> (new String[] { "A", "B", "C" });
```

9. JPanel - An area in which components can be placed and organized.

10. JFrame - A main window where components are added

```
JFrame frame = new JFrame("My window");
```

■ Event Handling

In Java GUI Programming (like Swing or AWT), Event Handling is the mechanism that controls the response to user actions such as clicking a button, typing in a text field, selecting an item from a combobox, closing a window and moving a mouse. Each user interaction generates an event and the program needs to handle that event to perform an appropriate action. This process known as event handling. However it involves three main components:

- Event Source - the GUI component (e.g. JTextField) that generates the events.
- Event object - contains information about the event.
- Event Listener - the object that listens for the event and defines the response.

How the Event handling works

steps: (How it got registered)

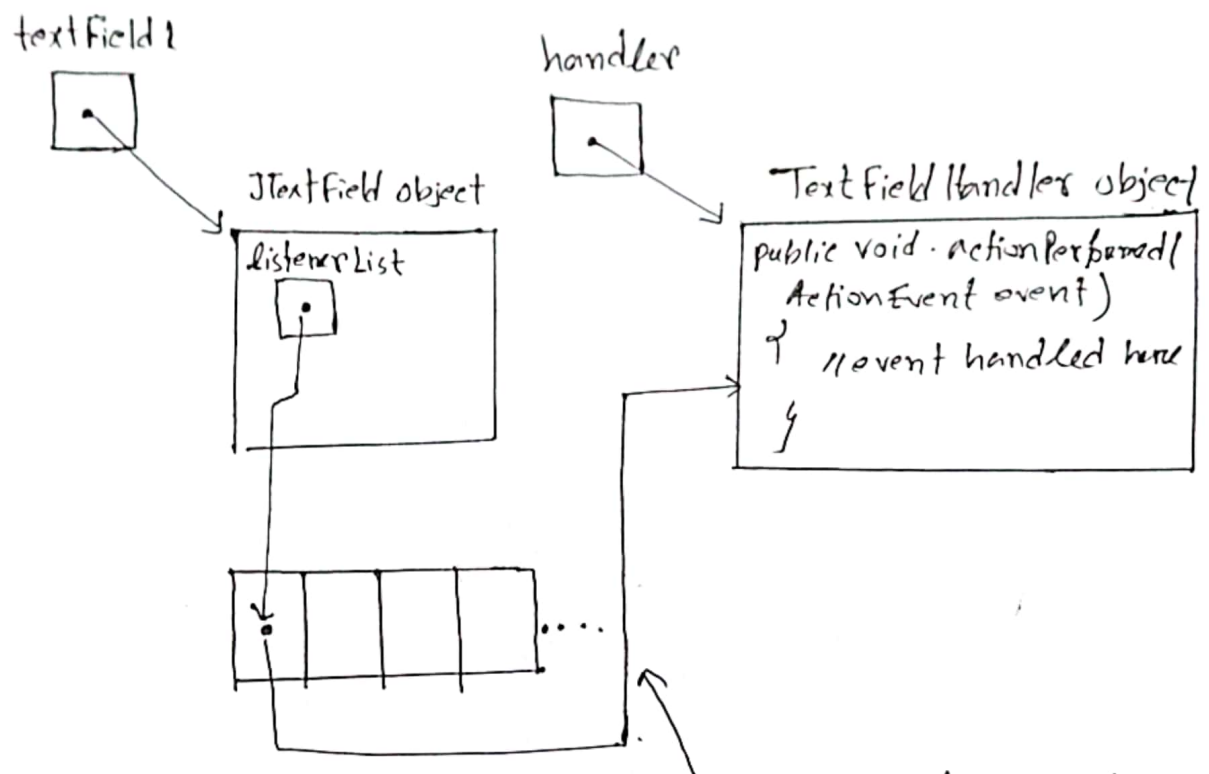
To make a component respond to an event, you must registered an event handler. For example:

```
textField.addActionListener(handler);
```

This line connects the textField1 component with a listener object that handles the action. Here handler is an object of a class that implements the ActionListener interface.

When this code runs, the handler is added to textField1's internal list of listeners (listenerList).

Behind the scenes, the list stores both the object and the type event it listens for (like ActionListener).



This reference is created by the statement `textField1.addActionListener(handler);`

Fig 2.1 : Event registration for `JTextField`

Here `textField1` is the source. `handler` is added to `textField1`'s `listenerList`. Now the component knows who to notify when an event occurs.

Step 2: (How does the GUI Component know which Method to call)

When an Event happens, like pressing Enter in text field:

- The Java Virtual Machine (JVM) generates an event object with an event ID (e.g., ActionEvent)
- The GUI component checks its listener list to find all listeners registered for that type of event
- It then calls appropriate method on each listener.

For example:

- ActionEvent → calls actionPerformed() method in all ActionListener objects.
- MouseEvent → calls method in MouseListener or MouseMotionListener.
- KeyEvent → calls methods in KeyListener.

Once you register a listener, the component automatically knows what event types it's handling & which method to call on the registered object.

Layout Manager

A Layout Manager in Java is used to arrange GUI components in a container for presentation purposes. Instead of specifying the exact size and position of each component, layout managers allow for more flexible and consistent UI design by handling most of the layout details automatically.

There are three main ways to arrange components in a GUI.

1. Absolute Positioning: Gives full control over component placement by setting the layout to null, but requires manual specification of position and size. This can be tedious without an IDE.
2. Layout Managers: Simplify GUI design by automatically positioning components.
3. Visual Programming in an IDE; Modern IDEs provide drag-and-drop design tools, enabling easy GUI creation and layout adjustment. They generate the underlying Java code and often support layout managers internally.

Some common layout managers are:

- FlowLayout
- BorderLayout
- GridLayout
- BoxLayout
- CardLayout.

① FlowLayout: The FlowLayout is used to arrange the components in a line, one after another (in a flow). It is the default layout of applet or panel. The fields of FlowLayout class are LEFT, RIGHT, CENTER, LEADING, TRAILING.

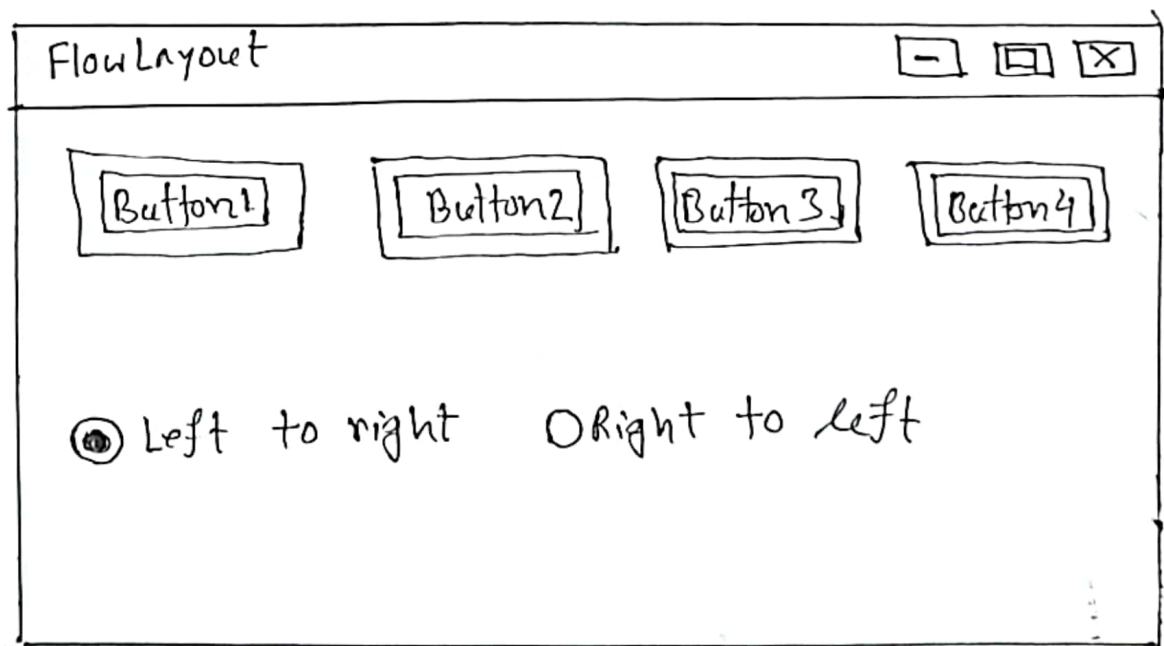


fig-1.1: Flowlayout frame.

Constructor of FlowLayout class :

- FlowLayout(): creates a flow layout with center alignment and a default 5 unit horizontal and vertical gap.
- FlowLayout(int align): create a flow layout with the given alignment and a default 5 unit horizontal and vertical gap.
- FlowLayout(int align, int hgap, int vgap): creates a Flow layout with the given alignment and the given horizontal and vertical gap.

② BorderLayout : The BorderLayout is used to arrange the components in five regions: north, south, east, west and center. Each region may contain one component only. It is the default layout of a frame or window. It provides five constants for each region:

NORTH, SOUTH, EAST, WEST, CENTER.

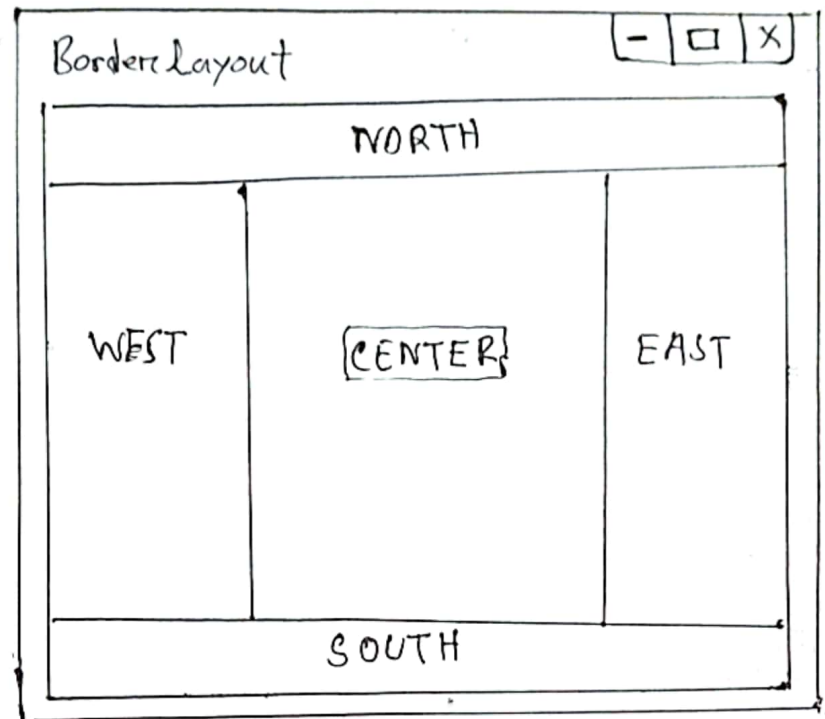
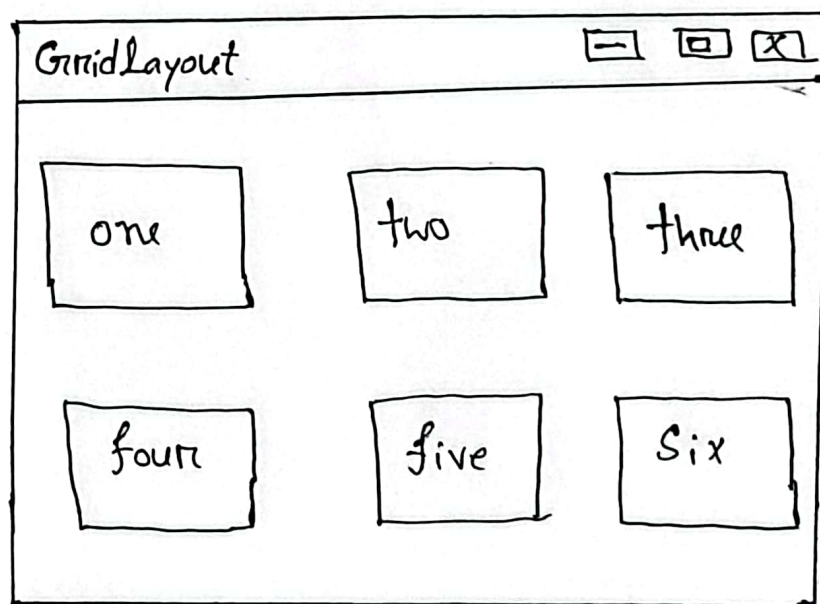


fig-1.2 : BorderLayout frame diagram

Constructors :

- BorderLayout() : creates a border layout but with no gaps between the components
- BorderLayout(int hgap, int vgap) : creates a border layout with the given horizontal and vertical gaps between the components

③ GridLayout : The GridLayout is useful to divide the container into a grid so that components can be placed in rows and columns. class GridLayout inherits directly from class object and implements interface LayoutManager. Components are added to a GridLayout starting at the top-left cell of the grid and proceeding left to right until the row is full. Then the process continues left to right on the next row of the grid and so on.



Constructors :

- . GridLayout() : creates a grid layout with one column per component in a row.

■ GridLayout(int rows, int columns): creates a grid layout with the given rows & columns but no gaps between the components.

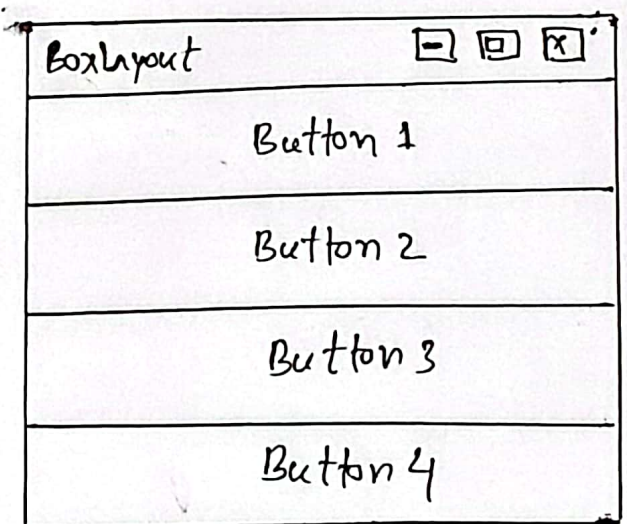
■ GridLayout(int rows, int columns, int hgap, int vgap): creates a grid layout with the given rows and columns along with given horizontal and vertical gaps.

④ The BorderLayout: The BorderLayout is used to arrange the components either vertically or horizontally. BorderLayout class found in javax.swing package.

constructor:

■ BorderLayout(Container c, int axis): creates a box layout that arranges the components with the given axis. This Axis are

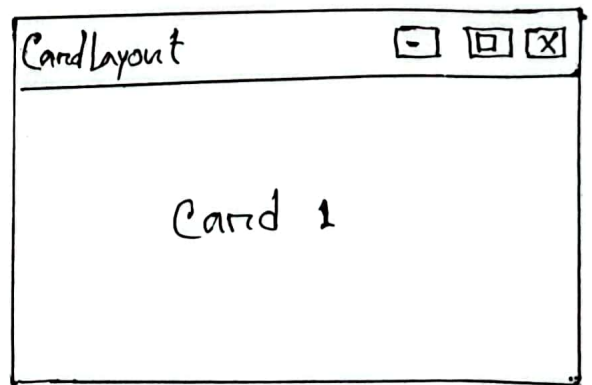
- X-Axis
- Y-Axis
- Line-Axis
- Page-Axis



⑤ CardLayout : The Java CardLayout class manages the components in such a manner that only one component is visible at a time. It treats each component as a card that's why it is known as CardLayout.

► Constructors :

▪ CardLayout() : creates a card layout with zero horizontal and vertical gap.



▪ CardLayout(int hgap, int vgap) :
creates a card layout with the given horizontal and vertical gap.

Methods :

- public void next(Container parent) : Flip to the next card of the given container.
- public void previous(Container parent) : Flip to the previous card of the given container.
- public void last(Container parent) : Flip to the last card of the given container.
- public void show(Container parent, String name) : Flip the specified card with the given name.

Exercise 14.1

Ans:

- Ⓐ Method mouseMoved is called when the mouse is moved with no buttons pressed and an event listener is registered to handle the event.
- Ⓑ Text that cannot be modified by the user is called uneditable (read-only) text.
- Ⓒ A(n) layout manager arranges GUI components in a container.
- Ⓓ The add method for attaching GUI components is a method of class Container.
- Ⓔ GUI is an acronym for graphical user interface.
- Ⓕ Method setLayout is used to specify the layout manager for a container.
- Ⓖ A mouseDragged method call is preceded by a(n) mousePressed, mouseReleased method call.
- Ⓗ Class JOptionPane contains methods that displays message dialogs and input dialogs.
- Ⓘ An input dialog capable of receiving input from the user is displayed with the method showInputDialog of class JOptionPane.
- Ⓢ A dialog capable of displaying a message to the user is displayed with method of class ShowMessageDialog, JOptionPane.

⑧ Both JTextFields and JTextAreas directly extend class JTextComponent.

Exercise 14.7

Ans :

① given, `import javax.swing.JFrame`

Error: Missing semicolon (;) at the end of the code

Corrected: `import javax.swing.JFrame ;`

In java, every import statement must end with a semicolon; otherwise it results compilation error.

② Given `panelObject.GridLayout(8,8);`

Error: GridLayout is a class, not a method. You can't call it like a method on panelObject.

Corrected:

`panelObject.setLayout(new GridLayout(8,8));`

To set a layout manager for a panel, you use `setLayout()` method & pass a new instance of `GridLayout`.

② given,

```
container.setLayout(new FlowLayout(FlowLayout.DEFAULT));
```

Error: There is no constant called FlowLayout.DEFAULT in the FlowLayout class.

corrected :

```
container.setLayout(new FlowLayout(FlowLayout.CENTER);
```

because valid alignment constants in FlowLayout

③ ~~Given~~ are :

FlowLayout.LEFT, FlowLayout.RIGHT, FlowLayout^{out}.CENTER
FlowLayout.LEADING & FlowLayout.TRAILING

④ Given,

```
container.add(eastButton, EAST);
```

Error: EAST is not defined directly. It should be BorderLayout.EAST.

Corrected: container.add(eastButton, BorderLayout.EAST);

Cause, In a BorderLayout, the position constants (eg., EAST, WEST, CENTER) are defined as static fields in the BorderLayout class.